

Operating systems

Memory Management

Content Lecture#11

01 Memory Management

02 Address Binding

03 Segmentation

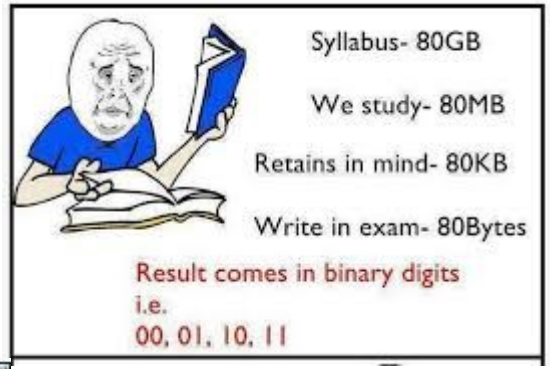
04 Paging

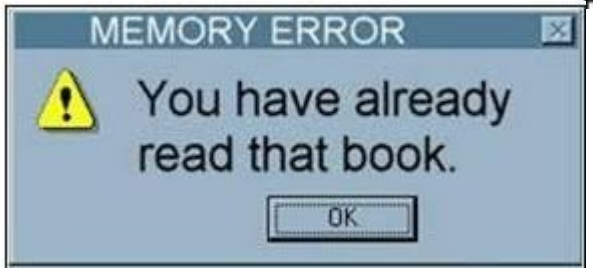
05 Address Translation

Lets take a Break

A programmer had a problem, so he used threds. Then he had two problems.









Memory

A

Address binding

is the mapping of the processor view of memory to the physical memory locations Typically occurs at one of three stages

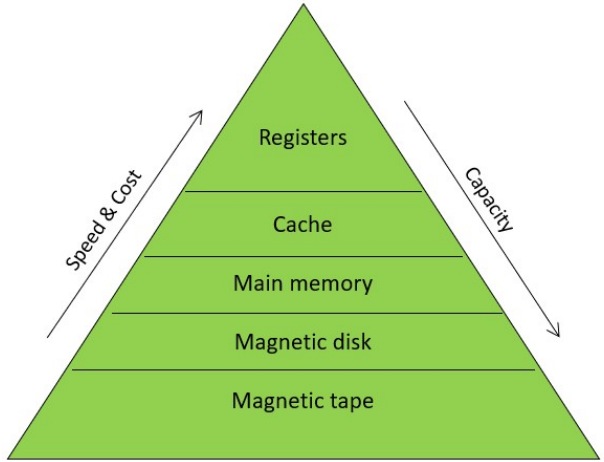
Memory

- Memory
  - Memory hierarchy
  - Process loading and swapping
  - Memory management
  - Memory allocation methods
  - Storage management strategies
  - Contiguous memory allocation
- Non contiguous memory allocation
  - Segmentation
    - Simple segmentation
    - Segment addressing
  - Paging
    - Simple paging
    - Implementation of paging

Memory **performance** largely depends upon

- The **organization** of memory
- Storage **capacity**
- **Speed** of memory system

- Used for storage and retrieval of data and instructions
- Essential component
- Logically memory system can be divided into three groups
- Internal memory
  - Set of registers in processor
  - they hold temporary results during execution
- Main memory
  - Communicates directly with CPU
- Secondary (external memory)
  - Peripherals storage devices i.e disk & tape etc



Memory Hierarchy

- **Registers**
  - are fastest, smallest and most expensive
- **Main memory**
  - It is principal memory
  - It is generally extended with a higher speed smaller cache
    - Used to improve movement of data between main memory and processor registers
    - Usually not visible to user or programmer or indeed to the processor
- **External devices are permanent storage**
  - Also referred as secondary or auxiliary memory
  - e.g. magnetic disk, tape, hard disk

- The fastest memory is closest to CPU and slowest is away
- The access time and approximate capacity for each memory is;

Name	Access Time	Capacity
Register	2-3 ns	<1KB
Cache	3-5 ns	4-6MB
Main memory	7-10 ns	2GB – 32GB
Disk	100 ms	500GB – 6TB

- Suppose processor request a word
  - If it is in level-1 cache, processor accesses it directly
  - If it is in level-2 cache the word is transferred to level-1 then to processer
  - Similarly data can be transferred from lower levels

Process Loading & Swapping

- Process loading
  - Transfer of a process from secondary storage to main memory
- Process Swapping
  - Transfer of a process from memory to secondary storage

Memory Management

- Memory is a volatile array of words or bytes, shared by the CPU and I/O devices
- Programs must be brought into memory and place in form of a process to be executed
- The operating system manages the memory activities
  - Keep track of which parts of memory are currently being used and by whom
  - Decides which process(s) to load when memory space becomes available
- Allocate and de-allocate memory space to processes as required In a uni-programming environment, main memory is divided into two parts
  - for operating system
  - for the user programs currently being executed
- In a multiprogramming environment, the user part is further subdivided into sub-parts to accommodate several processes
- The task of sub division of memory to pack as many processes is carried out dynamically by the memory manager and is called memory management
  - Memory manger is part of operating system

Memory Manager Duties &Problem

- **Duties**
  - •Execute maximum processes at the same time
  - •Provide satisfactory level of performance
  - •Sharing of memory space between processes
  - •Protect each process from other
  - •Transparent address of memory space for programmers

- **Problems** which can arise at programmer's level
  - •Insufficient main memory available; for a program plus its data
  - •The programmer must engage in a practice known as overlaying
    - •Keep in memory only those instructions and data that are needed at any given time
    - •Needed when process is larger than amount of memory allocated to it
    - •In this program and data are organized in various modules which can be assigned the same region of memory, with a main program responsible for switching modules in and out as need
  - •In a multiprogramming environment, programmer does not know how much space will be available or where the space will be

Address Binding

Address binding is the mapping of the processor view of memory to the physical memory locations
Typically occurs at one of three stages

- **Compile time**
  - If memory location is known priori, absolute code can be generated
  - Must recompile code if starting location changes
- **Load time**
  - Must generate re-locatable code if memory location is not known at compile time
- **Execution time**
  - Binding delayed until run time if process can be moved during its execution from one memory segment to another
    - Need hardware support for address mapping

Logical vs. Physical Address Space

Logical VS Physical

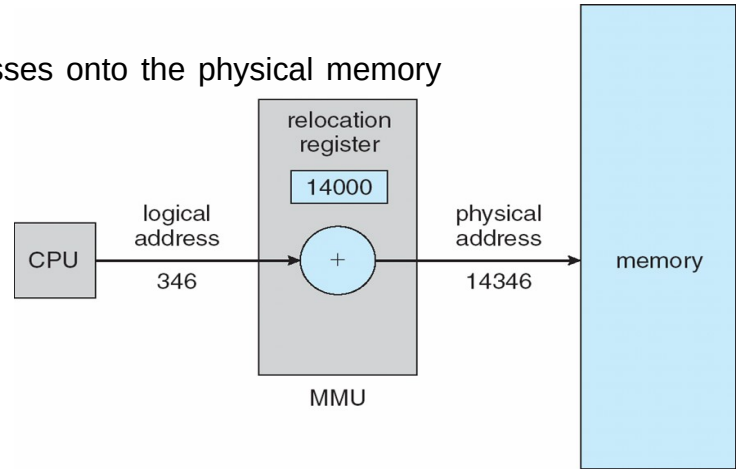
- Logical address
  - Generated by CPU
  - Also referred to as virtual address
- Physical address
  - Address seen by the memory unit
- Compile and load time binding
  - Logical = physical memory
- Execution time binding
  - Logical is virtual memory, i.e. does not necessarily coincide with physical memory

Virtual Addresses

- There exist a set of memory addresses that programs can produce
- These program generated addresses are called virtual or logical addresses and form virtual address space
- Physical addresses are the addresses seen by the memory unit
- Logical and physical addresses are same at compile time
- Logical and physical addresses differ in execution time

Memory Management Unit

- Hardware device that maps virtual address to physical address
- Use a base or relocation register
  - The value in relocation register is added to every address generated by a user process at the time it is sent to memory
  - The user programs deals with logical addresses
    - It never sees the real physical addresses
  - On computers without virtual memory, the virtual addresses are put directly onto memory bus
  - When virtual memory is used, the virtual addresses do not go directly to memory bus
    - Instead they go to memory management unit (MMU)
  - MMU is a chip or collection of chips that converts the virtual addresses onto the physical memory addresses
  - The user program deals with logical addresses
    - it never sees the real physical addresses



Memory Management Requirements

1. **Relocation**
   - Main memory is shared between processes
   - Swap active processes in and out to provide large pool of ready processes
     - Quite difficult to declare when swapped out process will be placed back
   - A process image occupies a contiguous region of main memory
   - To run this process, the processor hardware and operating system must be able to translate the memory reference found in code, into physical memory address

2. **Sharing**

   Allow several processes to access the same area of main memory

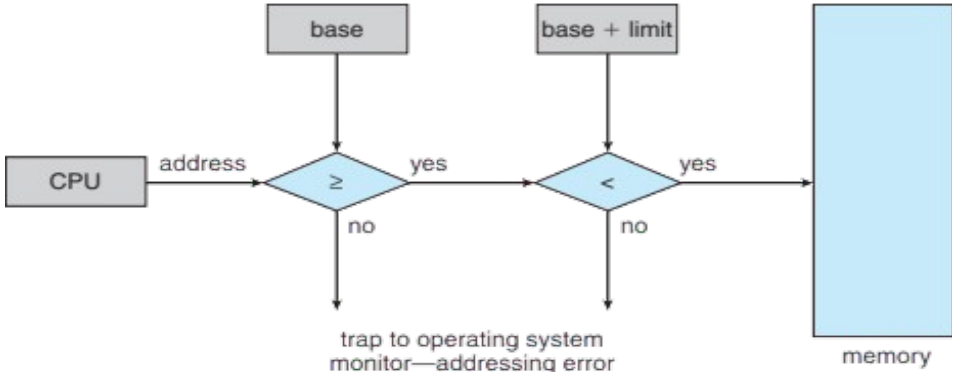
   Processes that are cooperating on some tasks may need to share access the same data structure

   Allow controlled access to share areas of main memory without compromising essential protection

Memory Management Requirements

3. **Protection**

- Each process must be protected by unwanted interference by other processes
- Without prior permission, a process should not refer to other process
- All memory references generated by a process must be checked at run time
  - to ensure that they refer only to the allocated memory space
  - The processor must abort such instructions when executed
- Thus the memory protection requirement must be satisfied by the processor (hardware) rather than operating system
  - Since OS cannot anticipate all the memory references a program will make
- Even if such participations are possible, it would be prohibitively time consuming to screen each program in advance for possible memory reference violation
- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



Memory Management Requirements

- **4. Local organization**
  - Main memory is organized as a linear or one dimensional, address space that consist of a sequence of bytes or words
  - Secondary memory at its physical level is similarly organized
  - It does not correspond to the way programs are constructed
  - Mostly programs are organized into modules
    - Some of which are un-modifiable (read or executable only)
    - Some of which contain that can be modified

5. Physical organization

- Secondary memory of large capacity provides long term storage of program and data
- While a smaller main memory holds programs and data currently in use
- The organization of flow of information between main memory and secondary memory is a major system concern

Program Loading into Memory

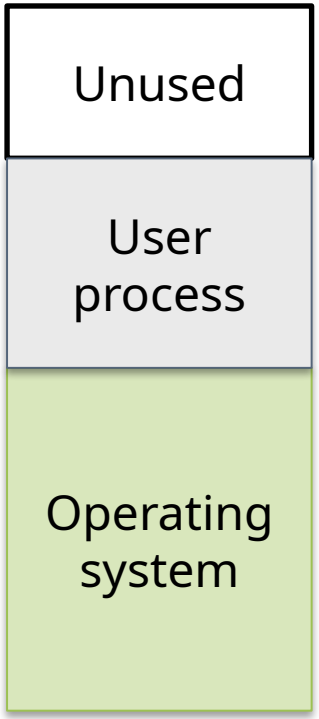
- The core task of any memory management system is to bring and manage programs in main memory
- In almost all modern multiprogramming systems, this task involves sophisticated scheme known as virtual memory
- Virtual memory is in turn based on the use of one or both of two basic techniques
    - Segmentation
    - Paging

Memory Management Techniques

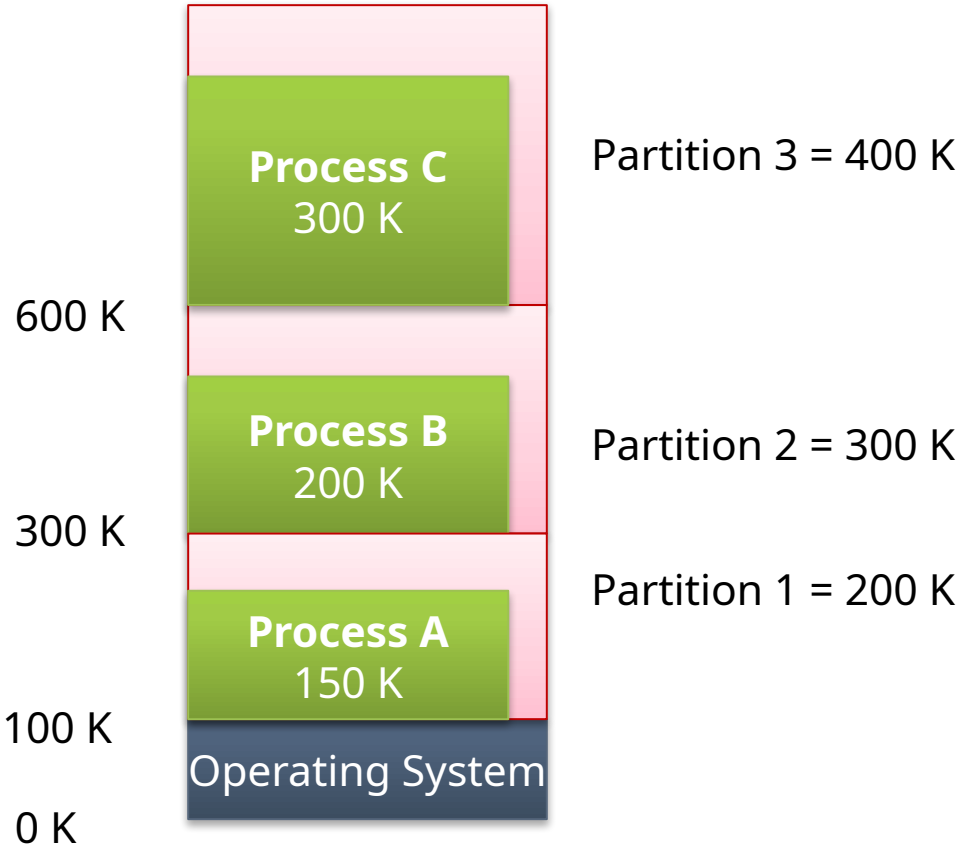
- Different schemes used in operating systems to allocate memory to processes
  - Contiguous Memory Allocation
    - Single process system
    - Fixed size (static) partition memory
    - Variable sized (dynamic) partition memory
    - Segmentation
  - Non Contiguous Memory Allocation
    - Paging
    - Paged segmentation
    - Demand paging
    - Demand segmentation
    - Virtual memory

Single Process System

- Memory is divided into two partitions
  - One for user process and
  - One for the operating system
- Most common to place the operating system in low memory
- Run one process at a time in high memory
- Memory management is very simple
- Example;
  - MS-DOS system is an example of this system



Fixed Sized Partition Memory



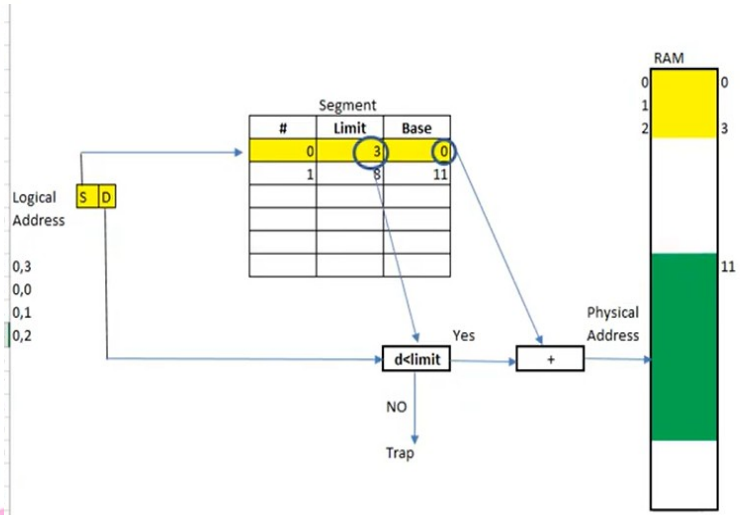
Variable-Sized Partition Memory

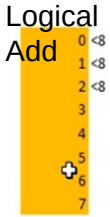
- The processes would be given as much storage as they required
- Processes are loaded into consecutive areas until the memory is filled
- When a partition is free another process is loaded into free partition
- External fragmentation
    - It refers to the distribution of the free memory space, out-with any process allocation
- What are the problems here?
    - What if the size of the vacant partition(s) is smaller than the size of the incoming process?
    - A new process cannot be started
- Solution
    - Coalescing
        - It refers to combine two or more adjacent holes to accommodate a large incoming process
        - Difficulty: Coalescing is a time consuming procedure
    - Variable partition scheme has been used successfully in many computers
    - Overheads
        - In variable partition, the system must keep track of
            - the position
            - size of each hole
            - taking account of the effect of coalescing
        - The solution is to use linked list, which contains
            - Pointer to the start of each hole and
            - The hole size

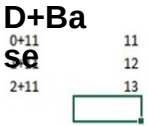
Variable-Sized Partition Memory

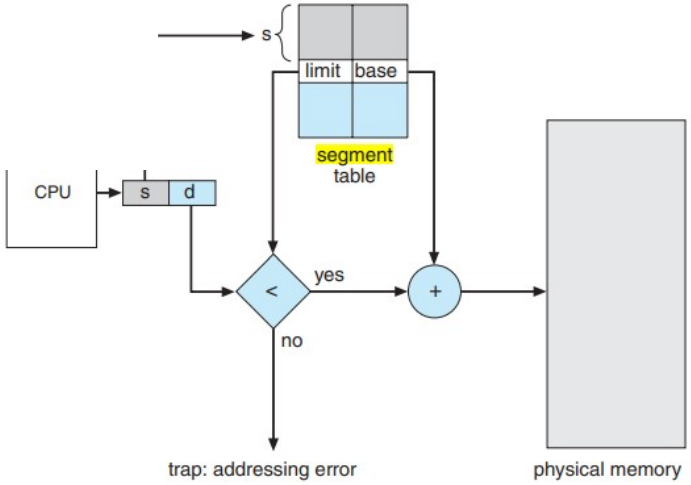
- Variable-sized partition with compaction
  - Compaction is a process of physically moving memory resident processes to close up the holes
  - Difficulty;
    - It is a time consuming procedure
  - All processes are suspended while reshuffling takes place
  - Not feasible in time critical system
  - Compaction may be performed
    - As soon as any process terminates
    - When new process cannot load due to fragmentation
    - At fixed intervals
    - When the user decide to do

Segmentation



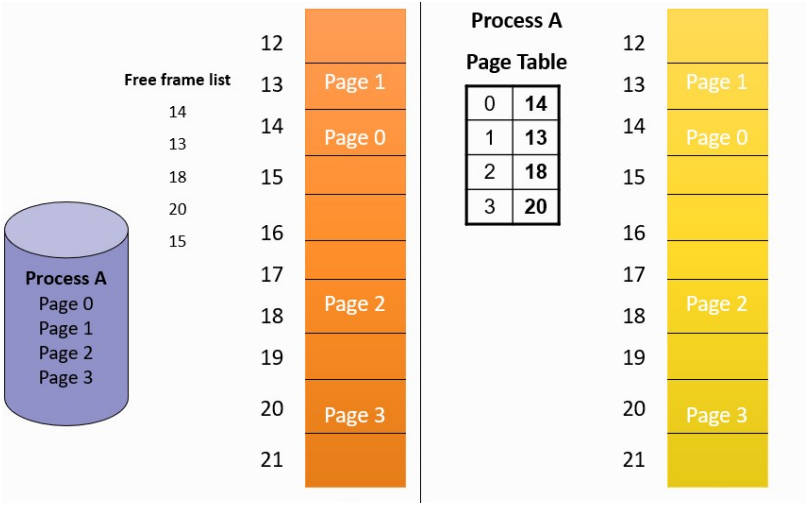






Paging

- Fixed sized and variable sized partitions are insufficient due to
  - Internal fragmentation
  - External fragmentation
- Paging removes these problems
  - Main memory is partitioned into equal fixed-size chunks called **frames**
  - Relatively small in size
  - Each process is also divided into fixed size chunks called **pages**
  - Same size as memory frames
  - Page size is typically power of 2 (typically 4K-8K)
    - depending on the computer architecture
  - No need of contiguous frames for a process or even in correct order

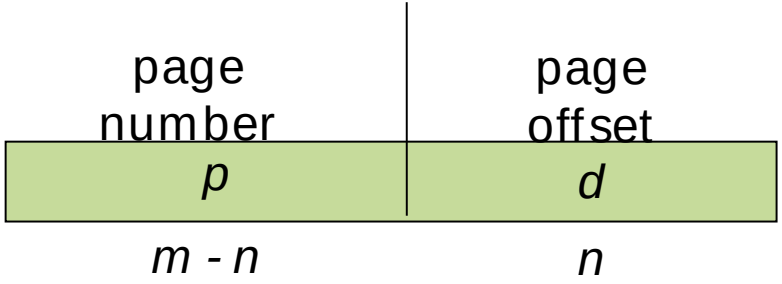


Page Addressing

- In paging, a memory address locations has the form (p,d)
  - Where **p** is number of the page
  - **d** is the displacement (or offset) of the location from the start of the page
- These parameters are derived from the actual memory address
- Consider an example of 16-bit address;
  - **p** - page no. has the value range 0 to 31 (2^5-1) or 32 pages
  - **d** – displacement value range 0 – 2047 i.e. (2^{11}-1)
  - This system would have 32 pages each of 2048 locations
  - Example;

	Page no	Displacement
Binary	01101	00000101100
Decimal	13	44

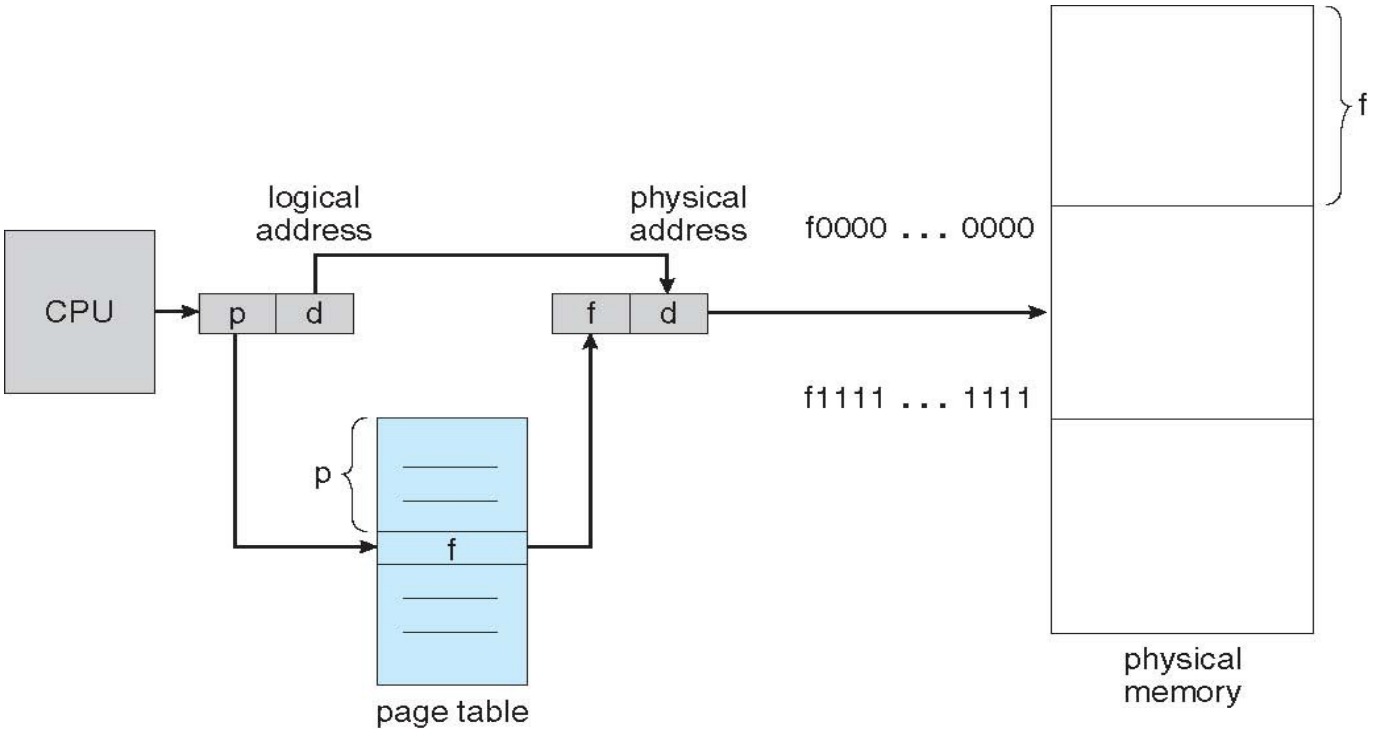
- Hence Page address is (13,44)



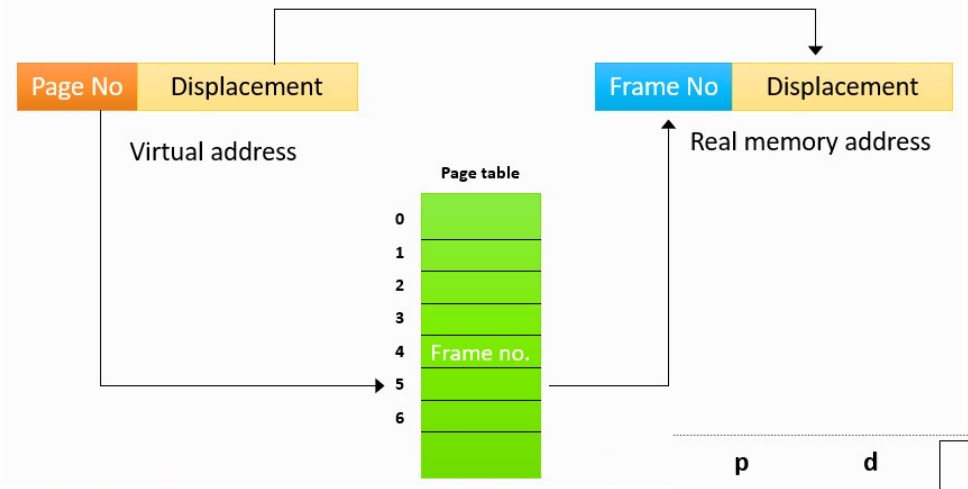
Page Address Translation

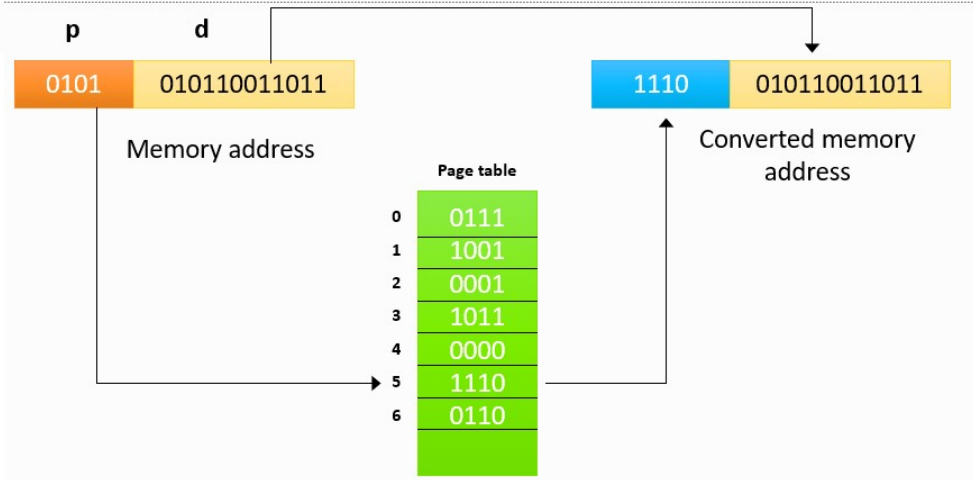
- Each process has its own page table
- The page tables are held in memory
- The page table register contains memory address of the page table for the active process
- A context switching also saves the value of page table register
- The address translation and page table maintenance are implemented in hardware called memory management unit (MMU) to achieve maximum speed of translation
- Each page table entry contains the real memory page translation and number of control bits
- E.g.
  - The present or p bit indicates whether corresponding page is in main memory or not
  - The modified or M bit indicate whether or not the page has been modified
    - A modified page is called dirty page
- When a page is positioned in a frame, the page number parameter of address changes but displacement remains constant
- The Intel Pentium can use 32 bit addressing space providing 4 gigabytes of addressable space

Page Address Translation

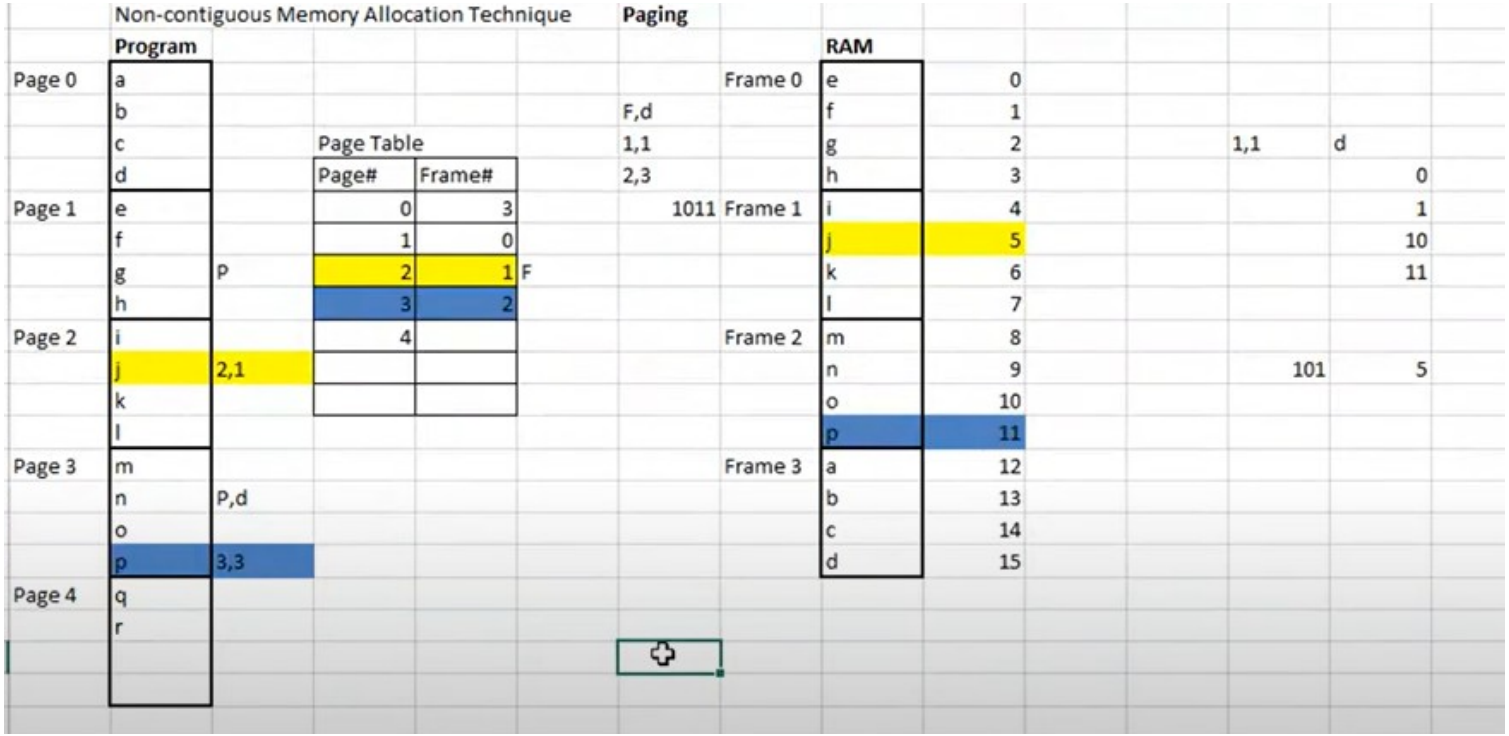


Page Address Translation





Page Address Translation

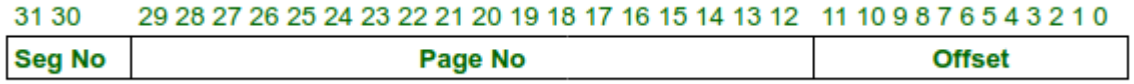


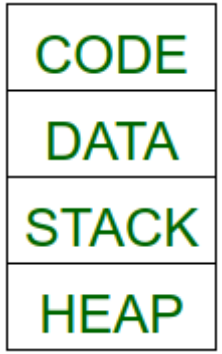
Paged Segmentation

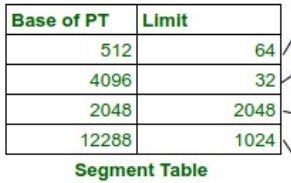
A solution to the problem is to use segmentation along with paging to reduce the size of page table. Traditionally, a program is divided into four segments, namely code segment, data segment, stack segment and heap segment.

The size of the page table can be reduced by creating a page table for each segment. To accomplish this hardware support is required. The address provided by CPU will now be partitioned into segment no p

The memory management unit (MMU) will use the segment table which will contain the address of page table(base) and limit. The page table will point to the page frames of the segments in main memory.

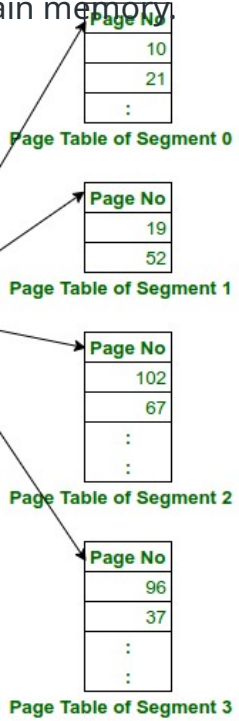






Base of PT	Limit
512	64
4096	32
2048	2048
12288	1024

Segment Table



Page No
10
21
:

Page Table of Segment 0

Page No
19
52

Page Table of Segment 1

Page No
102
67
:
:

Page Table of Segment 2

Page No
96
37
:
:

Page Table of Segment 3

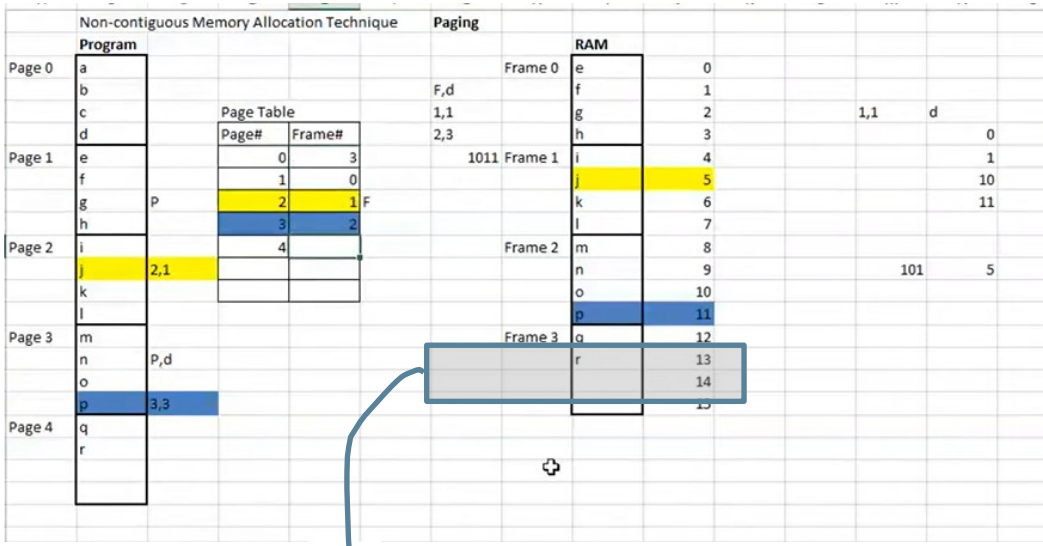
Internal Fragmentation

- Memory is divided into a number of fixed partitions
- Each partition may contain exactly one process
- When partition is free another process is loaded into free partition
- Waste considerable amount of main memory when processes are smaller than allocated partition
  - **Called internal fragmentation**
- First employed in multiprogramming computers
- Used in IBM OS/360

Disadvantages

Prevent a process being run due to unavailability of sufficient size partition

Internal fragmentation wastes space



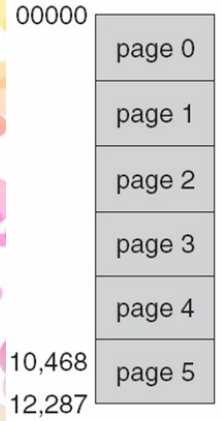
- Memory is allocated more than the requirements so this is called internal fragmentation

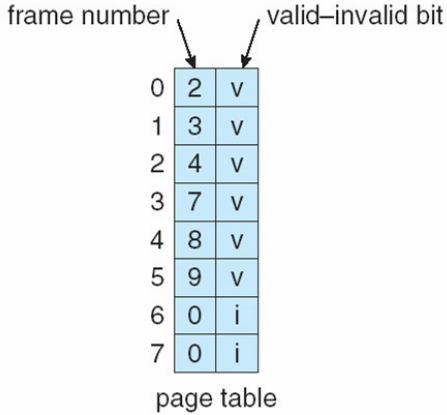
Protection

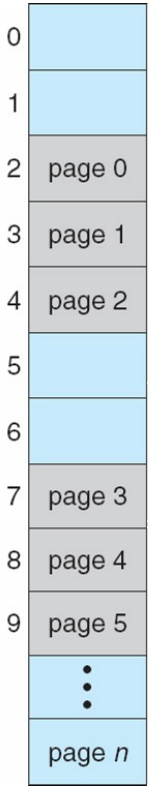
- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
  - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid bit** attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space
  - Or use page-table length register (PTLR)
- Any violations result in a trap to the kernel

  On checking valid bit: Trap for the page is not available or the page is not the part of program

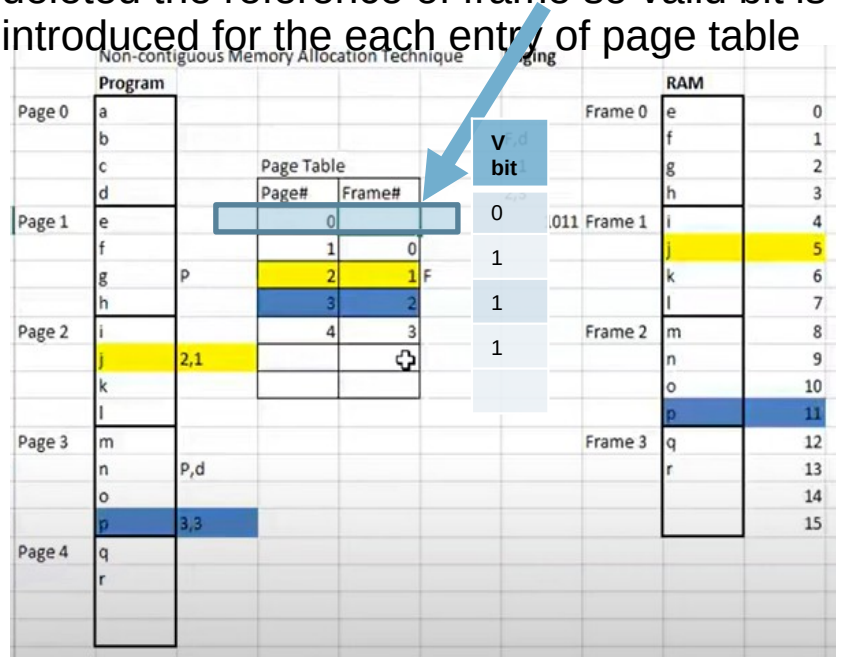
Protection



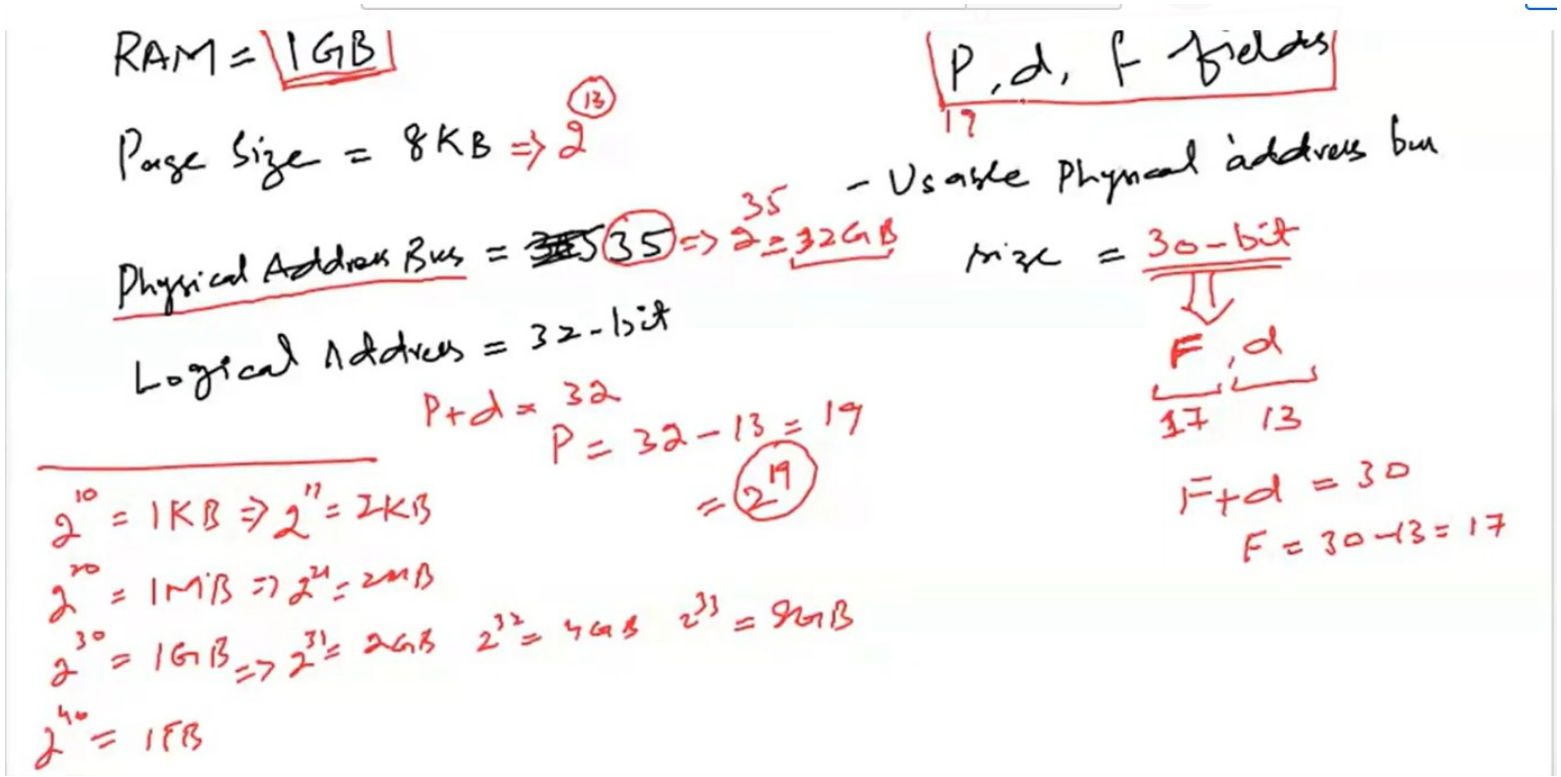




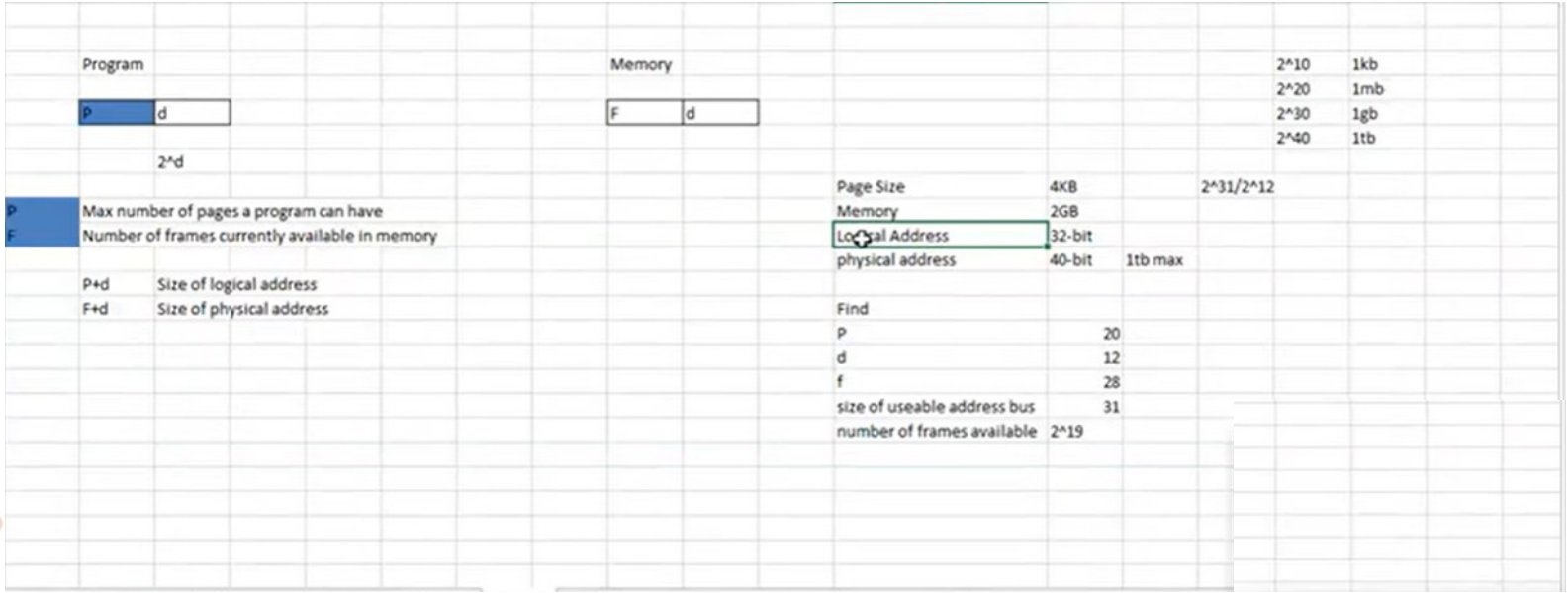
- Page zero is not available in memory so deleted the reference of frame so valid bit is introduced for the each entry of page table



Bits calculation



Bits calculation



Program			Memory					
p	d		F	d			2^10	1kb
							2^20	1mb
	2^d						2^30	1gb
							2^40	1tb
P	Max number of pages a program can have				Page Size	4KB	2^31/2^12	
F	Number of frames currently available in memory				Memory	2GB		
					Logical Address	32-bit		
P+d	Size of logical address				physical address	40-bit	1tb max	
F+d	Size of physical address							
					Find			
					P	20		
					d	12		
					f	28		
					size of useable address bus	31		
					number of frames available	2^19		



Thank you